UNINTER — Ensino a Distância (EaD)

Disciplina de Big Data

Sandbox Big Data

Guia do Aluno

Ambiente Hadoop completo em Docker para as aulas práticas 7 a 12 e para a Atividade Prática, em substituição à máquina virtual HDP 2.6.5.

Versão 1.0 — setembro de 2026
Prof. Guilherme Ditzel Patriota

Sumário

1. Introdução 4
1.1 O que você vai encontrar neste guia 4
1.2 Convenções usadas no texto 4
2. Requisitos e instalação 5
2.1 Requisitos 5
2.2 Windows 10 e 11 5
2.3 macOS 6
2.4 Linux 7
2.5 A primeira subida 7
3. Uso no dia a dia 8
3.1 Os três comandos 8
3.2 Outros subcomandos 9
3.3 Onde ficam os arquivos 10
4. O ambiente 11
4.1 Acessos e senhas 11
4.2 Interfaces web 11
4.3 O Ambari 12
4.4 Os dois Sparks 12
5. Roteiro das aulas práticas 13
5.1 Aula 7 — Pig e Oozie 13
5.2 Aula 8 — Hive, Sqoop, MapReduce, HBase, NoSQL, Drill e Presto 13
5.3 Aula 9 — Flume, Kafka, Storm e Flink 16
5.4 Aulas 10 e 11 — PySpark 18
5.5 Aula 12 — Spark Streaming e Structured Streaming 18
6. A Atividade Prática 20
6.1 Opção A — Google Colab 20
6.2 Opção B — Sandbox Docker 21
7. Memória e boas práticas 23
7.1 Perfis de memória 8g e 16g 23
7.2 Não deixe os serviços ligados por horas a fio 23
7.3 Sinais de falta de memória 24
8. Problemas comuns 25
Apêndice A — Versões dos componentes 26
Apêndice B — Errata dos materiais originais 27
Apêndice C — O que é igual e o que é diferente da VM HDP 28
Igual 28

Diferente 28

Este sumário é atualizado automaticamente pelo Word. Se aparecer vazio, clique com o botão direito sobre ele e escolha “Atualizar campo”.

1. Introdução

A disciplina de Big Data usa, nas aulas práticas, a máquina virtual Hortonworks HDP 2.6.5. Essa máquina foi descontinuada: o download não está mais disponível, os repositórios de pacotes deixaram de existir, o boot leva de trinta a sessenta minutos (e cerca de doze horas em computadores Apple com chip M1/M2/M3) e ela exige 8 GB de memória só para si.

A **Sandbox Big Data** substitui essa máquina virtual por um ambiente Docker. Ela mantém o que os vídeos das aulas mostram (o Ambari, o usuário `maria_dev`, o nome `sandbox.hortonworks.com`, as mesmas portas e senhas) usando componentes atuais e mantidos: Apache Ambari 2.7.11 e a distribuição ODP 1.2.4, que é a continuação em código aberto do HDP.

A diferença mais importante para quem tem um computador modesto: em vez de ligar todos os serviços de uma vez, a sandbox liga **somente os serviços que cada aula usa**. Por isso ela roda em computadores com 8 GB de RAM, embora 16 GB ou mais tornem o uso mais confortável.

1.1 O que você vai encontrar neste guia

- Como instalar o Docker e a sandbox no Windows, macOS ou Linux (capítulo 2).
- Como ligar a sandbox, preparar uma aula e acessar o Ambari, o terminal e as demais interfaces (capítulos 3 e 4).
- Um roteiro para cada aula prática, indicando exatamente o que muda em relação ao vídeo (capítulo 5).
- Como fazer a Atividade Prática, no Google Colab ou na sandbox, à sua escolha (capítulo 6).
- Orientações sobre memória, boas práticas e solução de problemas (capítulos 7 e 8).
- Apêndices com a tabela de versões, a errata dos materiais originais e a lista de diferenças para a VM HDP.

1.2 Convenções usadas no texto

Comandos e nomes de arquivos aparecem em fonte monoespaçada, como `lab aula 7`. Blocos de comando aparecem em caixas cinza; digite-os linha a linha. Comandos que começam com `.\lab.ps1` são executados no PowerShell do **seu computador**, dentro da pasta do projeto. Comandos como `lab`, `hdfs`, `pig` ou `spark-submit` são executados **dentro da sandbox** (no terminal web ou por SSH), como usuário `maria_dev`.

> **Windows, macOS e Linux**
> Os blocos de comando mostram sempre as duas formas: `.\lab.ps1 ...` no PowerShell do Windows e `./lab.sh ...` no terminal do macOS ou do Linux. São o mesmo utilitário, com os mesmos subcomandos; use a linha do seu sistema.

2. Requisitos e instalação

2.1 Requisitos

Item	Mínimo	Recomendado
Memória RAM	8 GB	16 GB ou mais
Processador	2 núcleos, 64 bits, com virtualização ativada	4 núcleos ou mais
Espaço em disco livre	40 GB (a imagem ocupa cerca de 22 GB)	60 GB
Sistema operacional	Windows 10/11 com WSL2, macOS 12 ou superior, Linux	—
Internet	Download único da imagem (vários GB)	—

Computadores Apple com chip M1, M2 ou M3 funcionam por emulação (Rosetta). A sandbox fica mais lenta nesses aparelhos, mas leva minutos para subir, e não as doze horas da máquina virtual antiga.

2.2 Windows 10 e 11

Siga os passos na ordem. Cada um tem o link da documentação oficial, caso queira mais detalhes ou capturas de tela.

Passo 1 — Confirmar que a virtualização está ativa

Abra o **Gerenciador de Tarefas** (Ctrl+Shift+Esc), aba **Desempenho**, item **CPU**. No canto inferior direito procure “Virtualização: Habilitado”. Se estiver “Desabilitado”, é preciso ligar a opção na BIOS/UEFI do computador (o nome varia: Intel VT-x, Intel Virtualization Technology, AMD-V ou SVM Mode). O passo a passo da Microsoft, com os caminhos das marcas mais comuns, está em Habilitar a virtualização no Windows.

Passo 2 — Instalar o WSL2

Clique no menu Iniciar, digite **PowerShell**, clique com o botão direito em “Windows PowerShell” e escolha **Executar como administrador**. Na janela azul, digite o comando abaixo e pressione Enter. Ao terminar, reinicie o computador.

```
wsl --install
```

Na primeira abertura após reiniciar, o Ubuntu pede para criar um usuário e uma senha do Linux; anote-os, mas eles não são usados pela sandbox. Documentação oficial: Instalar o WSL — Microsoft Learn. Para conferir se está tudo certo, execute `wsl --status`: deve aparecer “Versão padrão: 2”.

Passo 3 — Instalar o Docker Desktop

Baixe o instalador em Docker Desktop para Windows (botão “Docker Desktop for Windows – x86_64”) e execute-o. Mantenha marcada a opção **Use WSL 2 instead of Hyper-V**. Ao final, abra o Docker Desktop pelo menu Iniciar e espere o ícone da baleia, no canto inferior da janela, ficar estável (sem

animação). Não é necessário criar conta; feche a tela de login clicando em “Skip” ou “Continue without signing in”.

Passo 4 — Limitar a memória do WSL2

Sem esse limite o WSL2 pode tomar metade da RAM do computador e travar o Windows. Abra o **Bloco de Notas**, cole o conteúdo abaixo e salve como `.wslconfig` (com o ponto na frente e sem `.txt` no fim) dentro da sua pasta de usuário, que é `C:\Users\<seu nome de usuário>`. No diálogo Salvar, troque “Tipo” para “Todos os arquivos” para o Windows não acrescentar `.txt`. Se o computador tiver 16 GB de RAM, use `memory=10GB`.

```
[wsl2]
memory=6GB
swap=6GB
processors=2
autoMemoryReclaim=gradual
```

Em seguida, no PowerShell, execute `wsl --shutdown` e abra o Docker Desktop novamente para o limite passar a valer. Referência: Configurações avançadas do WSL.

Passo 5 — Baixar e extrair a sandbox

Baixe o arquivo ZIP do projeto na plataforma da disciplina ou em github.com/N-CPUninter/bigdata-lab-docker, pelo botão verde **Code > Download ZIP**. Clique com o botão direito no arquivo baixado e escolha **Extrair Tudo...**. Na janela que abre, apague o caminho sugerido e digite `C:\bigdata-lab`, depois clique em **Extrair**. O caminho não pode ter espaços nem acentos; evite pastas como “Área de Trabalho” ou “Documentos” dentro do OneDrive.

Passo 6 — Subir a sandbox

Abra o Explorador de Arquivos em `C:\bigdata-lab` (a pasta onde estão os arquivos `lab.ps1` e `docker-compose.yml`), clique na barra de endereço, digite `powershell` e pressione Enter: um PowerShell abre já dentro da pasta. Execute:

```
.\lab.ps1 up
```

Se aparecer a mensagem “a execução de scripts foi desabilitada neste sistema”, execute uma única vez o comando abaixo, responda `S` e repita o `.\lab.ps1 up`:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

2.3 macOS

1. Baixe e instale o Docker Desktop pelo guia oficial Docker Desktop para Mac, escolhendo o instalador do seu chip (Apple Silicon ou Intel). Arraste o ícone para a pasta Aplicativos e abra-o.
2. Em **Settings > Resources**, mova o controle de memória para pelo menos 6 GB (10 GB se o Mac tiver 16 GB ou mais) e clique em “Apply”.

3. Mac com chip Apple (M1, M2, M3): em **Settings > General**, marque “Use Rosetta for x86_64/amd64 emulation on Apple Silicon”. A imagem da sandbox é x86_64. Se o macOS pedir para instalar o Rosetta, aceite. Detalhes em Docker Desktop no Apple Silicon.
4. Baixe o ZIP do projeto pela plataforma ou em github.com/N-CPUninter/bigdata-lab-docker, dê dois cliques para extrair e mova a pasta para um local sem espaços no nome (por exemplo `~/bigdata-lab`).
5. Abra o **Terminal** (Launchpad > Outros > Terminal), entre na pasta com `cd ~/bigdata-lab` e execute `./lab.sh up`. Se aparecer “permission denied”, execute antes `chmod +x lab.sh`.

2.4 Linux

1. Instale o Docker Engine e o plugin Compose pelo guia da sua distribuição em Install Docker Engine (Ubuntu, Debian, Fedora etc.). Ao final, `docker compose version` precisa responder.
2. Adicione seu usuário ao grupo `docker` para não precisar de `sudo`: `sudo usermod -aG docker $USER`, depois saia e entre novamente na sessão. Referência: Linux post-installation steps.
3. Extraia o ZIP do projeto (`unzip bigdata-lab.zip -d ~/bigdata-lab`), entre na pasta e execute `./lab.sh up`.

2.5 A primeira subida

Na primeira execução de `.\lab.ps1 up` (ou `./lab.sh up`) o Docker baixa a imagem da sandbox. O tempo depende da sua conexão. Em seguida o Ambari leva de dois a quatro minutos para responder; o script espera por isso, mostra as URLs de acesso e indica o próximo passo: `aula <perfil>`. A partir da segunda vez a sandbox sobe em cerca de um minuto.

Subir a sandbox **não** liga os serviços das aulas. Só o Ambari fica no ar. É o comando `aula`, descrito no capítulo 3, que liga o HDFS, o YARN, o Hive e os demais serviços de cada aula, e ele termina com a mensagem `PRONTO`. Até lá, não tente rodar comandos das aulas.

> **Atualizações**
> Quando o professor publicar uma nova versão da imagem, execute `docker compose pull` dentro da pasta do projeto e depois `.\lab.ps1 up` (ou `./lab.sh up`). Seus dados (HDFS, bancos, arquivos da `maria_dev`) ficam em volumes do Docker e sobrevivem à atualização.

3. Uso no dia a dia

3.1 Os três comandos

Tudo o que você precisa no dia a dia cabe em três comandos, executados no seu computador, dentro da pasta do projeto. À esquerda a forma do Windows (PowerShell); à direita a do Linux e do macOS.

Windows (PowerShell)	Linux / macOS (terminal)	O que faz
`.\lab.ps1 up`	`./lab.sh up`	1. Sobe a sandbox e espera o Ambari responder
`.\lab.ps1 aula 7`	`./lab.sh aula 7`	2. Prepara a aula 7: liga só os serviços dela e termina com `PRONTO`
`.\lab.ps1 shell`	`./lab.sh shell`	3. Abre um terminal dentro da sandbox, como `maria_dev`

O subcomando `aula` é o coração da sandbox: ele **desliga** os serviços que a aula não usa e **liga** os que ela usa, esperando cada um ficar de pé. O argumento que você passa a ele é o **perfil**: o número da aula (`7`, `10`, `11`, `12`), uma parte de uma aula grande (`8a`, `8b`, `9c`), a palavra `trabalho` para a Atividade Prática ou `none` para desligar tudo. Alguns exemplos:

Windows (PowerShell)	Linux / macOS (terminal)	Resultado
`.\lab.ps1 aula 10`	`./lab.sh aula 10`	Aula 10 (PySpark): liga ZooKeeper e HDFS
`.\lab.ps1 aula 11`	`./lab.sh aula 11`	Aula 11: os mesmos serviços da 10; se você veio da aula 10, nada muda
`.\lab.ps1 aula trabalho`	`./lab.sh aula trabalho`	Atividade Prática: ZooKeeper e HDFS, para o upload pelo Files View
`.\lab.ps1 aula 8a`	`./lab.sh aula 8a`	Primeira parte da aula 8: HDFS, YARN, MapReduce2 e Hive
`.\lab.ps1 aula none`	`./lab.sh aula none`	Fim dos estudos: desliga todos os serviços; só o Ambari fica no ar

> **Espere a mensagem PRONTO**
> Ligar os serviços leva de um a seis minutos, conforme o perfil e o computador. Enquanto isso o comando mostra o que está ligando e, a cada 30 segundos, o tempo decorrido. Ao final ele imprime uma faixa com `PRONTO em XmYYs`, a lista dos serviços de pé e as URLs.
>
> **Só depois dessa mensagem** abra o terminal web, conecte por SSH ou rode os comandos das aulas. Antes dela, `hdfs`, `hive`, `pig`, `spark-submit` e as interfaces web falham ou ficam presos, porque os serviços ainda estão subindo. Se tiver dúvida, `.\lab.ps1 status` (ou `./lab.sh status`) mostra o que está `STARTED`; no Ambari, todos os serviços do perfil devem estar verdes.

Os perfis disponíveis estão na tabela a seguir.

Perfil	Serviços ligados	Conteúdo
7	ZooKeeper, HDFS, YARN, MapReduce2, Hive, Oozie	Pig e Oozie
7a	ZooKeeper, HDFS, YARN, MapReduce2, Hive	Só Pig (mais leve)
8a	ZooKeeper, HDFS, YARN, MapReduce2, Hive	Hive, Sqoop/MariaDB, MapReduce com mrjob
8b	ZooKeeper, HDFS, YARN, MapReduce2, HBase	HBase (e Pig com HBaseStorage)
8c	ZooKeeper, HDFS + containers Mongo e Cassandra	MongoDB, Cassandra e Spark 2
8d	ZooKeeper, HDFS, Hive + Mongo/Cassandra	Drill e Presto
9a	ZooKeeper, HDFS	Flume
9b	ZooKeeper, Kafka	Kafka e Kafka Connect
9c	ZooKeeper (+ Storm por `lab storm start`)	Storm
9d	nenhum	Flink (standalone)
10, 11	ZooKeeper, HDFS	PySpark: RDD, DataFrame, Spark SQL, MLlib
12	ZooKeeper, HDFS	Spark Streaming e Structured Streaming
trabalho	ZooKeeper, HDFS	Atividade Prática (upload + spark-submit)
none	nenhum	Desliga tudo; só o Ambari fica no ar

3.2 Outros subcomandos

Windows	Linux / macOS	O que faz
`.\lab.ps1 status`	`./lab.sh status`	Serviços ligados no Ambari e memória em uso
`.\lab.ps1 stop-all`	`./lab.sh stop-all`	Desliga todos os serviços (o container continua)
`.\lab.ps1 down`	`./lab.sh down`	Para o container; dados preservados nos volumes
`.\lab.ps1 nosql`	`./lab.sh nosql`	Sobe os containers MongoDB e Cassandra (aulas 8c e 8d)
`.\lab.ps1 mem 16g`	`./lab.sh mem 16g`	Perfil de memória para máquinas com 16 GB ou mais (padrão: `8g`)
`.\lab.ps1 logs`	`./lab.sh logs`	Mostra o log da sandbox
`.\lab.ps1 reset`	`./lab.sh reset`	Apaga tudo (HDFS, bancos, arquivos) e volta ao estado inicial. Pede confirmação

Dentro da sandbox o mesmo utilitário existe como `lab`: `lab aula 8a`, `lab status`, `lab storm start`, `lab urls` e assim por diante.

> **Ao terminar de estudar**
> Execute `.\lab.ps1 stop-all` ou `.\lab.ps1 down` (`./lab.sh stop-all` ou `./lab.sh down`). Os serviços consomem cada vez mais memória se ficarem ligados por muitas horas, mesmo sem uso, e o computador fica lento. O capítulo 7 explica o motivo.

3.3 Onde ficam os arquivos

A pasta do projeto no seu computador tem duas subpastas que aparecem dentro da sandbox:

- `aulas/` — os arquivos de cada aula (scripts Pig, Python, configurações do Flume etc.), já com os erros de digitação do material original corrigidos. Dentro da sandbox: `/home/maria_dev/aulas`.
- `data/` — os conjuntos de dados (`ml-100k`, `customer-orders.csv`, `access_log.txt`, `movielens.sql`). Dentro da sandbox: `/home/maria_dev/data-host`. Uma cópia dos dados pequenos já existe em `/home/maria_dev/data`, e o `ml-100k` já está carregado no HDFS em `/user/maria_dev/ml-100k`.

Arquivos que você criar no home da `maria_dev` dentro da sandbox ficam guardados em um volume do Docker e persistem entre reinicializações, até um `reset`.

4. O ambiente

4.1 Acessos e senhas

Acesso	Endereço	Usuário / senha
Ambari	http://localhost:8080	admin/admin, raj_ops/raj_ops ou maria_dev/maria_dev
Terminal no navegador	http://localhost:4200	maria_dev/maria_dev
SSH (PuTTY ou terminal)	ssh -p 2222 maria_dev@localhost	maria_dev
Página com todos os links	http://localhost:8888	—
MariaDB (o MySQL das aulas)	mysql -u root -phadoop dentro da sandbox	root/hadoop
Root do Linux da sandbox	.\lab.ps1 root ou ./lab.sh root	root/hadoop

O terminal web e o SSH aceitam login assim que a sandbox sobe, mas os comandos das aulas só funcionam depois que o `aula <perfil>` imprimir `PRONTO`. No PuTTY, informe Host Name `localhost`, Port `2222`, Connection type SSH, e faça login como `maria_dev`. O nome do host continua sendo `sandbox.hortonworks.com`, o HDFS responde em `hdfs://sandbox.hortonworks.com:8020` e o ResourceManager na porta 8050. Por isso os arquivos de configuração das aulas (como o `job.properties` do Oozie) funcionam sem alteração.

4.2 Interfaces web

Interface	Endereço	Quando está disponível
Ambari (serviços, Files View, Workflow Manager)	http://localhost:8080	Sempre
HDFS — NameNode	http://localhost:9870	Perfis com HDFS
YARN — ResourceManager	http://localhost:8088	Perfis com YARN
MapReduce — histórico de jobs	http://localhost:19888	Perfis com MapReduce2
HiveServer2	http://localhost:10002	Perfis com Hive
Oozie	http://localhost:11000/oozie	Perfil 7
HBase Master	http://localhost:16010	Perfil 8b
Storm UI	http://localhost:8744	Após `lab storm start`
Flink	http://localhost:8082	Após `start-cluster.sh` (aula 9)
Drill / Presto	http://localhost:8756 / http://localhost:8090	Após iniciá-los (aula 8d)

Interface	Endereço	Quando está disponível
Spark UI do job em execução	http://localhost:4040	Enquanto um `spark-submit` roda
Zeppelin (notebooks)	http://localhost:9995	Após `lab start ZEPPELIN`

> **Porta do NameNode**
> Nos vídeos a interface do NameNode aparece na porta 50070. Dentro da sandbox ela continua nessa porta, mas no seu navegador o endereço é `localhost:9870`, porque o Windows reserva a faixa 50000–50959 para uso interno.

4.3 O Ambari

O Ambari é o mesmo painel dos vídeos: mostra o estado de cada serviço, permite ligar e desligar pelo menu "Service Actions" e traz o **Files View** (menu de nove pontos no canto superior direito), usado para enviar arquivos ao HDFS. Duas views que existiam no HDP 2.6.5 não estão mais disponíveis: o **Hive View** (o Ambari 2.7 o removeu) e o **Pig View** (depende de um componente que a distribuição ODP não inclui). Nos dois casos o guia de cada aula mostra como fazer a mesma coisa pelo terminal.

Você pode ligar e desligar serviços pelo Ambari, exatamente como o professor faz nos vídeos. O comando `lab aula` apenas automatiza essa sequência e cuida da ordem correta.

4.4 Os dois Sparks

Como no HDP, a sandbox tem duas versões do Spark, escolhidas pela variável `SPARK_MAJOR_VERSION`. O Python correto é selecionado automaticamente.

	Spark 2	Spark 3
Versão	2.4.8 (Scala 2.12)	3.5.1 (Scala 2.12)
Selecionar	`export SPARK_MAJOR_VERSION=2`	`export SPARK_MAJOR_VERSION=3` (padrão)
Python	3.7 (automático)	3.9 (automático)
Quando usar	Aula 12 (Spark Streaming com Flume) e para reproduzir o vídeo	Aulas 10 e 11 e a Atividade Prática

Os scripts das aulas rodam nas duas versões. Atenção a um detalhe herdado do HDP: `sc.textFile("u.data")` sem caminho absoluto lê do **HDFS** (`/user/maria_dev/u.data`), não da pasta atual. Para ler um arquivo local use o prefixo `file://`, por exemplo `file:///home/maria_dev/aulas/aula10/01-RatingsCounter/u.data`.

5. Roteiro das aulas práticas

Cada seção começa pelo comando que prepara a sandbox e depois segue a ordem do vídeo, apontando o que mudou. Os arquivos citados estão em `~/aulas/aulaNN` dentro da sandbox.

5.1 Aula 7 — Pig e Oozie

```
.\lab.ps1 aula 7              # Windows — ou aula 7a (só Pig, mais leve)
./lab.sh aula 7               # Linux / macOS — ou aula 7a (só Pig, mais leve)
```

O `ml-100k` já está no HDFS em `/user/maria_dev/ml-100k`. O vídeo mostra o envio pelo Files View; você pode repetir para praticar, a pasta é a mesma.

Pig

O Pig View não está disponível; use o terminal, como o vídeo faz na segunda parte. O `pig` roda no Tez sobre o YARN lendo do HDFS. Existe também o modo local (`pig -x local`), que lê do disco: nesse caso troque `/user/maria_dev/ml-100k` por `/home/maria_dev/data/ml-100k` no script.

```
cd ~/aulas/aula07/Pig_exercise
pig MostRatedFiveStarMovie.pig
pig OldestFiveStarMovie.pig
```

Correções feitas no material original: `ORDER finalResults BY numRatings DESC` (a forma `... AS numRatings DESC` não é Pig Latin válido) e `videoRelease` (estava `videoRealese`).

Oozie

Os arquivos `job.properties`, `workflow.xml` e `oldmovies.sql` funcionam sem alteração. O banco `movielens` já está carregado no MariaDB, então a ação Sqoop do workflow encontra a tabela `movies`.

```
cd ~/aulas/aula07/Oozie_exercise
hdfs dfs -put -f workflow.xml oldmovies.sql /user/maria_dev/
oozie job -oozie http://localhost:11000/oozie -config job.properties -run
oozie job -oozie http://localhost:11000/oozie -info <ID retornado>
hdfs dfs -cat /user/maria_dev/oldmovies/*
```

Acompanhe em `http://localhost:11000/oozie` ou no Ambari, em **Workflow Manager** (uma view nova, que não existia no HDP 2.6.5). O workflow leva de três a cinco minutos. No log aparece um aviso de que a classe `com.mysql.jdbc.Driver` está obsoleta; pode ignorar.

5.2 Aula 8 — Hive, Sqoop, MapReduce, HBase, NoSQL, Drill e Presto

A aula 8 usa muitos serviços e não cabe inteira em 8 GB. Ela foi dividida em quatro perfis; prepare um de cada vez.

Perfil	Comando	Temas
8a	`.\lab.ps1 aula 8a` / `./lab.sh aula 8a`	Hive, Sqoop e MariaDB, MapReduce com mrjob

Perfil	Comando	Temas
8b	`.\lab.ps1 aula 8b` / `./lab.sh aula 8b`	HBase e Pig com HBaseStorage
8c	`.\lab.ps1 nosql` e depois `.\lab.ps1 aula 8c` (Linux/macOS: `./lab.sh nosql`, `./lab.sh aula 8c`)	MongoDB, Cassandra e Spark 2
8d	`.\lab.ps1 nosql` e depois `.\lab.ps1 aula 8d` (Linux/macOS: `./lab.sh nosql`, `./lab.sh aula 8d`)	Drill e Presto

8a — Hive

No HDP as tabelas eram criadas pelo "Upload Table" do Hive View, que não existe mais. O `lab aula 8a` cria automaticamente as tabelas `ratings` e `names` com o mesmo layout do vídeo. Para consultar, use o `beeline` (o comando `hive` também o abre):

```
beeline -u jdbc:hive2://localhost:10000 -n maria_dev
SHOW TABLES;
SELECT * FROM ratings LIMIT 5;
!run /home/maria_dev/aulas/aula08/Hive_example/MostPopularMovie.sql
```

Alternativa com interface web: o Zeppelin (`lab start ZEPPELIN`, depois `http://localhost:9995`), com o interpretador `%jdbc(hive)`. Ele consome cerca de 1 GB de RAM.

8a — Sqoop e MariaDB

O MySQL do HDP virou MariaDB, com o mesmo usuário `root` e senha `hadoop`. O banco `movielens` já está carregado. O acesso anônimo ao banco está liberado, como no HDP, então os comandos do vídeo, sem usuário e senha, também funcionam.

```
sqoop import --connect jdbc:mysql://localhost/movielens \
  --driver com.mysql.jdbc.Driver --username root --password hadoop \
  --table movies -m 1

sqoop import --connect jdbc:mysql://localhost/movielens \
  --driver com.mysql.jdbc.Driver --username root --password hadoop \
  --table movies -m 1 --hive-import
```

O material original trazia o erro de digitação `.com.mysql.jdbc.Driver` (um ponto a mais no início).

8a — MapReduce com mrjob

O `mrjob 0.7.4` já está instalado no Python 3. O código do exercício é o mesmo do vídeo.

```
cd ~/aulas/aula08
# local:
python MapReduce_exercise.py ~/data/ml-100k/u.data
# no YARN:
python MapReduce_exercise.py -r hadoop \
  --hadoop-streaming-jar \
  /usr/odp/current/hadoop-mapreduce-client/hadoop-streaming.jar \
  hdfs:///user/maria_dev/ml-100k/u.data
```

8b — HBase

```
hbase shell
  create 'users', 'userinfo'
  list
  exit
cd ~/aulas/aula08/HBase_example && pig hbase.pig
hbase shell
  count 'users'
  scan 'users', {LIMIT => 5}
```

> **Phoenix — limitação conhecida**
> O Phoenix 5.1.3 que vem na distribuição ODP 1.2.4 não é compatível com o HBase 2.5.6 da mesma distribuição: a conexão falha tanto no `sqlline.py` quanto no Query Server. Enquanto não houver um client compatível, estude o Phoenix pela leitura do `Phoenix.txt` (com o DDL corrigido) e pratique o acesso ao HBase pelo `hbase shell`, pelo Pig e pelo Spark.

8c — MongoDB e Cassandra

No HDP o Mongo era instalado como serviço do Ambari e o Cassandra por `yum`; os dois repositórios não existem mais. Na sandbox eles são dois containers oficiais que compartilham a rede da sandbox e continuam respondendo em `127.0.0.1`, de modo que os scripts `MongoSpark.py` e `CassandraSpark.py` rodam sem alteração.

```
.\lab.ps1 nosql              # Windows — sobe mongo e cassandra
./lab.sh nosql               # Linux / macOS — sobe mongo e cassandra
```

Dentro da sandbox, com o Spark 2 (os pacotes já estão em cache, nada é baixado):

```
export SPARK_MAJOR_VERSION=2
cd ~/aulas/aula08/MongoDB
spark-submit --packages org.mongodb.spark:mongo-spark-connector_2.12:2.4.4 MongoSpark.py
cd ~/aulas/aula08/Cassandra
spark-submit \
  --packages com.datastax.spark:spark-cassandra-connector_2.12:2.5.2 \
  CassandraSpark.py
```

Antes do `CassandraSpark.py` crie o keyspace e a tabela (comandos do `installing.txt`). Os shells rodam **no seu computador**, não dentro da sandbox:

```
docker compose exec mongo mongo          # shell do Mongo
docker compose exec cassandra cqlsh      # cqlsh
```

No log do `CassandraSpark.py` aparece um erro `NoClassDefFoundError: jnr/posix/POSIXHandler`; ele vem de um componente opcional do driver e não impede a gravação. Confira com `SELECT COUNT(*) FROM movielens.users` no `cqlsh`.

8d — Drill e Presto

Os pacotes já estão baixados em `~/downloads`, não é preciso `wget`. As versões são Drill 1.21.2 (a 1.12 do vídeo não conversa com o Hive 3) e Presto 0.235.1 (a mesma do vídeo).

```
cd ~ && tar -xzf downloads/apache-drill-1.21.2.tar.gz && cd apache-drill-1.21.2
bin/drillbit.sh start -Ddrill.exec.port=8756      # UI: http://localhost:8756
bin/drillbit.sh stop
```

No Drill, em Storage > hive > Update, informe `"hive.metastore.uris": "thrift://localhost:9083"`, como no vídeo.

```
cd ~ && tar -xzf downloads/presto-server-0.235.1.tar.gz && cd presto-server-0.235.1
cp -r ~/downloads/presto-etc/etc .                 # config da aula, heap reduzido
cp ~/downloads/presto-cli-0.235.1-executable.jar bin/presto && chmod +x bin/presto
bin/launcher start                                 # UI: http://localhost:8090
bin/presto --server 127.0.0.1:8090 --catalog hive
bin/launcher stop
```

5.3 Aula 9 — Flume, Kafka, Storm e Flink

Perfil	Comando	Tema
9a	`.\lab.ps1 aula 9a` / `./lab.sh aula 9a`	Flume
9b	`.\lab.ps1 aula 9b` / `./lab.sh aula 9b`	Kafka
9c	`.\lab.ps1 aula 9c` / `./lab.sh aula 9c` e, na sandbox, `lab storm start`	Storm
9d	`.\lab.ps1 aula 9d` / `./lab.sh aula 9d`	Flink

Os temas de Hive, HBase, Mongo, Cassandra, Drill e Presto da aula 9 são os mesmos da aula 8; o Spark Streaming está na aula 12.

Flume (9a)

O Flume 1.11 está em `/usr/hdp/current/flume-server`, como no vídeo. As pastas `~/spool` e `~/checkpoint` já existem.

```
cp ~/aulas/aula09/Flume/*.conf ~/
cd /usr/hdp/current/flume-server/
bin/flume-ng agent --conf conf --conf-file ~/example.conf --name a1 -
Dflume.root.logger=INFO,console
# em outro terminal:
nc localhost 44444        # ou: telnet localhost 44444
```

Para o exemplo spooldir → HDFS (`flumelogs.conf`), crie a pasta `/user/maria_dev/flume` pelo Files View (ou `hdfs dfs -mkdir /user/maria_dev/flume`) e copie `cp ~/data/access_log_small.txt ~/spool/log.txt`.

Kafka (9b)

O Kafka é o 3.6 (o HDP tinha o 1.0). A porta do broker continua 6667. A mudança que importa: as ferramentas não aceitam mais `--zookeeper`; use `--bootstrap-server`.

No vídeo	Na sandbox
`kafka-topics.sh --create --zookeeper sandbox.hortonworks.com:2181 ...`	`kafka-topics.sh --create --bootstrap-server localhost:6667 --replication-factor 1 --partitions 1 --topic Teste`
`kafka-topics.sh --list --zookeeper ...`	`kafka-topics.sh --list --bootstrap-server localhost:6667`
`kafka-console-producer.sh --broker-list ...:6667 --topic Teste`	Igual (`--broker-list` ainda funciona)
`kafka-console-consumer.sh --bootstrap-server ...:6667 --zookeeper localhost:2181 --topic Teste --from-beginning`	Remova `--zookeeper localhost:2181`
`./kafka-standalone.sh ...`	`./connect-standalone.sh ~/connect-standalone.properties ~/connect-file-source.properties ~/connect-file-sink.properties`

Os scripts ficam em `/usr/hdp/current/kafka-broker/bin` e os arquivos `.properties` de exemplo em `/usr/hdp/current/kafka-broker/config` (no HDP a pasta chamava `conf`).

Storm (9c)

O Storm 2.8.9 é instalado à parte do Ambari. Em vez de "Ambari > Storm > Start", use o comando abaixo dentro da sandbox. O restante é igual ao vídeo.

```
lab storm start          # nimbus, supervisor e UI (http://localhost:8744); aguarde ~30 s
storm jar /usr/hdp/current/storm-client/contrib/storm-starter/storm-starter-topologies-*.jar \
  org.apache.storm.starter.WordCountTopology wordcount
storm list
storm kill wordcount
lab storm stop
```

O código-fonte do exemplo está em `/usr/hdp/current/storm-client/examples/storm-starter/src/jvm/org/apache/storm/starter/WordCountTopology.java`; `contrib` é um atalho para `examples`. Os logs ficam em `/var/log/storm/workers-artifacts`.

Flink (9d)

O Flink 1.20 já está em `~/downloads` (o 1.10 do vídeo não roda no Java atual). Dois comandos mudaram de nome: `start-local.sh` virou `start-cluster.sh` e a porta da interface passa a ser definida por `rest.port` no arquivo `conf/config.yaml`.

```
cd ~ && tar -xzf downloads/flink-1.20.5-bin-scala_2.12.tgz && cd flink-1.20.5
grep -q '^rest.port' conf/config.yaml \
  && sed -i 's/^rest.port: .*/rest.port: 8082/' conf/config.yaml \
  || echo 'rest.port: 8082' >> conf/config.yaml
./bin/start-cluster.sh                      # UI: http://localhost:8082
./bin/flink run examples/streaming/SocketWindowWordCount.jar --port 9000
# em outro terminal:  nc -l 9000   e digite palavras
tail -f log/flink-*-taskexecutor-*.out
./bin/stop-cluster.sh
```

5.4 Aulas 10 e 11 — PySpark

```
.\lab.ps1 aula 10            # Windows
./lab.sh aula 10             # Linux / macOS
```

Liga apenas ZooKeeper e HDFS; o Spark roda em modo local. Como no HDP, os scripts leem `u.data`, `u.item` e `customer-orders.csv` do **HDFS** (`/user/maria_dev`). No vídeo o professor envia esses arquivos pelo Files View; na sandbox eles já estão lá, e você pode repetir o envio para praticar. Os `open("u.item")` do Python leem o arquivo local da pasta do script.

```
cd ~/aulas/aula10/01-RatingsCounter && spark-submit ratings-counter.py
cd ~/aulas/aula10/02-CustomerOrder
spark-submit customer-orders.py && spark-submit customer-orders-ordered.py
cd ~/aulas/aula10/03-MostPopularMovies
spark-submit popular-movies.py && spark-submit popular-movies-names.py
spark-submit popular-movies-dataframe.py
cd ~/aulas/aula10/04-SparkSQL && spark-submit spark-sql.py
cd ~/aulas/aula10/05-MLLib && spark-submit movie-recommendations-als.py 50
```

- Os scripts rodam no Spark 3 (padrão) e no Spark 2 (`export SPARK_MAJOR_VERSION=2`).
- `04-SparkSQL` lê `customer-order.csv`, no singular; a pasta já traz o arquivo com esse nome (no material da aula 10 o nome estava no plural e o script falhava; a aula 11 corrigiu).
- `05-MLLib` cria a pasta `checkpoint` no diretório atual.
- A aula 11 repete a aula 10; a pasta `aula11` tem os mesmos arquivos.

5.5 Aula 12 — Spark Streaming e Structured Streaming

```
.\lab.ps1 aula 12            # Windows
./lab.sh aula 12             # Linux / macOS
```

O exercício Flume → Spark exige o **Spark 2**: o pacote `spark-streaming-flume` não existe no Spark 3. A sandbox mantém o Spark 2.4.8 justamente para isso. Abra três terminais (o terminal web abre quantas abas quiser).

```
# terminal 1 — Spark
cd ~/aulas/aula12/SparkStreaming
export SPARK_MAJOR_VERSION=2
spark-submit --packages org.apache.spark:spark-streaming-flume_2.12:2.4.8 SparkFlume.py
```

```
# terminal 2 — Flume com sink Avro na porta 9092
cp ~/aulas/aula12/SparkStreaming/sparkstreamingflume.conf ~/
cd /usr/hdp/current/flume-server/
bin/flume-ng agent --conf conf --conf-file ~/sparkstreamingflume.conf --name a1
```

```
# terminal 3 — alimentar o spool
cp ~/data/access_log.txt ~/spool/log.txt
cp ~/data/access_log.txt ~/spool/log2.txt      # a contagem dobra
```

Diferença do vídeo: o pacote `spark-streaming-flume_2.11:2.0.0` virou `spark-streaming-flume_2.12:2.4.8`. O `SparkFlume2.py` (contagem por status HTTP) roda do mesmo jeito. Para o Structured Streaming, a pasta `logs` lida por `spark.readStream.text("logs")` fica no HDFS:

```
cd ~/aulas/aula12/SparkStreaming
hdfs dfs -mkdir -p logs
spark-submit structured-streaming.py
# em outro terminal:  hdfs dfs -put ~/data/access_log.txt logs/   (ou pelo Files View)
```

6. A Atividade Prática

O Roteiro da Atividade Prática aceita várias formas de execução: Google Colab, máquina virtual, containers Docker, nuvem ou Spark instalado localmente. **A escolha da plataforma é sua.** As duas formas prontas para uso são descritas abaixo; as regras do roteiro valem para qualquer uma delas: sem pandas, prints com o seu RU visível no comando e no nome das variáveis, entrega em PDF.

	Google Colab	Sandbox Docker
Instalação	Nenhuma; só uma conta Google	Docker Desktop + esta sandbox (capítulo 2)
Onde roda	No servidor do Google, pelo navegador	No seu computador
Memória necessária	Qualquer computador	8 GB de RAM
HDFS e Ambari	Não há (arquivo local no Colab)	Sim: upload pelo Files View, como nos vídeos
Versão do Spark	PySpark 3.5 instalado via `pip`	Spark 3.5.1 (`SPARK_MAJOR_VERSION=3`)
Indicado para	Quem quer o caminho mais leve e rápido	Quem quer praticar o fluxo Hadoop completo

6.1 Opção A — Google Colab

O notebook da atividade está publicado no GitHub da disciplina. Abra-o diretamente no Colab pelo link abaixo (é preciso estar logado em uma conta Google) e use **Arquivo > Salvar uma cópia no Drive** para ter a sua versão editável:

- Notebook no Colab: trabalho_big_data.ipynb
- Repositório da disciplina (notebook, dados e instruções): github.com/N-CPUninter/Big_Data

O notebook já traz as células que baixam o dataset, instalam o PySpark e criam a `SparkSession`; você preenche as células das funções `map` e `reduceByKey` de cada questão e executa. As primeiras células fazem, em resumo:

```
!wget https://raw.githubusercontent.com/N-CPUninter/Big_Data/main/data/imdb-reviews-pt-
br.zip \
      -O imdb-reviews-pt-br.zip
!unzip imdb-reviews-pt-br.zip
!pip install pyspark

from pyspark.sql import SparkSession
spark = (SparkSession.builder.appName('PySpark Trabalho de Big Data')
         .master('local').getOrCreate())
imdb_df = spark.read.csv('imdb-reviews-pt-br.csv', header=True,
                         quote='"', escape='"', encoding='UTF-8')
```

> **Sessão do Colab**
> O Colab desliga a máquina após algum tempo sem uso e apaga os arquivos baixados. Se isso acontecer, execute as células do início novamente (download, unzip, pip). O código que você escreveu fica salvo no notebook.

6.2 Opção B — Sandbox Docker

```
.\lab.ps1 aula trabalho       # Windows — liga ZooKeeper e HDFS
./lab.sh aula trabalho        # Linux / macOS — liga ZooKeeper e HDFS
```

Colocar o dataset na sandbox

1. Baixe o dataset: imdb-reviews-pt-br.zip (o mesmo arquivo usado pelo notebook do Colab; também disponível na plataforma). Extraia o ZIP e copie o `imdb-reviews-pt-br.csv` (cerca de 127 MB) para a pasta `data\` do projeto no seu computador, por exemplo `C:\bigdata-lab\data\`. Ele aparece dentro da sandbox em `/home/maria_dev/data-host/`.
2. Envie o arquivo ao HDFS pelo Ambari, como nos vídeos: abra `http://localhost:8080`, entre com `maria_dev` / `maria_dev`, clique no menu de nove pontos no canto superior direito e escolha **Files View**. Navegue até `user` > `maria_dev`, clique em **Upload**, selecione o CSV em `data-host` e confirme. O equivalente por comando, no terminal da sandbox, é `hdfs dfs -put ~/data-host/imdb-reviews-pt-br.csv /user/maria_dev/`.
3. Abra o terminal da sandbox (`http://localhost:4200`, usuário `maria_dev`) e confira o arquivo, trazendo-o também para o disco local como pede o roteiro:

```
hadoop fs -ls                                          # lista /user/maria_dev
hdfs dfs -copyToLocal /user/maria_dev/imdb-reviews-pt-br.csv .
grep -A1 fs.defaultFS /etc/hadoop/conf/core-site.xml   # ...:8020
```

Executar o script

Use o Spark 3 (é o padrão e é a mesma versão do PySpark do Colab). Há um esqueleto pronto em `~/aulas/trabalho/trabalho_RU.py` com as funções a preencher:

```
cp ~/aulas/trabalho/trabalho_RU.py ~/trabalho_SEU_RU.py   # renomeie com o seu RU
nano ~/trabalho_SEU_RU.py                                  # preencha as funções (Ctrl+O
salva, Ctrl+X sai)
spark-submit ~/trabalho_SEU_RU.py                          # print com o RU visível
```

> **Onde o Spark procura o arquivo**
> `spark.read.csv('imdb-reviews-pt-br.csv')` lê do **HDFS** (`/user/maria_dev`), como no HDP; por isso o passo do Files View. Para ler o arquivo local trazido pelo `copyToLocal`, use `file:///home/maria_dev/imdb-reviews-pt-br.csv`.

Também é possível usar o `pyspark` interativo, digitando `pyspark` no terminal e depois `imdb_df = spark.read.csv('imdb-reviews-pt-br.csv', header=True, quote='"', escape='"', encoding='UTF-8')`. Os parâmetros `quote` e `escape` são obrigatórios nas duas plataformas: sem eles o texto em português "vaza" para a coluna `sentiment`, o erro mais comum nas entregas.

Conferir

O script abaixo executa o seu programa e verifica se as respostas caem na faixa esperada. Ele não substitui as regras do roteiro; serve para você ter certeza de que a leitura do CSV está correta.

```
bash ~/aulas/trabalho/verifica.sh ~/trabalho_SEU_RU.py
```

7. Memória e boas práticas

A máquina virtual HDP precisava de 8 GB só para ela. Na sandbox o Ambari liga apenas os serviços de cada aula, e serviço parado não consome memória. A tabela mostra o consumo aproximado do container em cada perfil, com o perfil de memória padrão (`8g`).

Perfil	Serviços	Memória aproximada
base (sempre)	Ambari, PostgreSQL, MariaDB, sshd, terminal web	1,0 a 1,3 GB
9d	nenhum (Flink standalone)	2 GB
10, 11, 12, trabalho, 9a	ZooKeeper, HDFS (+ Spark local)	2,5 GB
9b	ZooKeeper, Kafka	2,3 GB
9c	ZooKeeper + Storm	3 GB
8c	ZooKeeper, HDFS + Mongo (0,5 GB) + Cassandra (1,5 GB)	4,5 GB
8d	ZooKeeper, HDFS, Hive + Mongo/Cassandra	4,5 GB
8a, 7a, 8b	ZooKeeper, HDFS, YARN, MapReduce2, Hive ou HBase	4,8 GB
7	8a + Oozie	5,5 GB

7.1 Perfis de memória 8g e 16g

Com 8 GB no computador, mantenha o perfil padrão (`8g`) e o `.wslconfig` com `memory=6GB`. Os perfis 7, 8a e 8d ficam lentos, mas funcionam; feche o navegador com muitas abas. Com 16 GB ou mais, dê 10 GB ao Docker e execute `.\lab.ps1 mem 16g` (ou `./lab.sh mem 16g`): os serviços ganham heaps maiores e os jobs ficam mais rápidos. O perfil `16g` é o mesmo para 16, 32 ou 64 GB de RAM; não existe um perfil maior nem é preciso ajustar mais nada.

	Perfil `8g`	Perfil `16g`
Heap do Oozie	384 MB	512 MB
Timeline Server e Timeline Reader do YARN	Ficam parados (o `lab aula` os desliga após ligar o YARN)	Ligados

O que se perde sem o Timeline no perfil `8g`: a interface do YARN (`localhost:8088`) não lista as aplicações **concluídas** que rodaram em Tez (Pig e Hive). O comando `yarn logs -applicationId` e o histórico do MapReduce (`localhost:19888`) continuam funcionando. No Ambari esses dois componentes aparecem como “Stopped”; é o esperado.

7.2 Não deixe os serviços ligados por horas a fio

Medição real com o perfil 7: logo após ligar, o container usa cerca de 5,4 GB; depois de um dia inteiro ligado, mesmo sem uso, chega a 7 GB (o limite), o processador fica ocupado e o Ambari para de responder. Os processos Java (Oozie, HiveServer2, Metastore) crescem lentamente com o tempo, e o cache de disco do HDFS também conta no limite do container.

> **Regra prática**
> Ao terminar a sessão de estudo, execute `lab aula none` (para todos os serviços e mantém o Ambari) ou `.\lab.ps1 down` / `./lab.sh down` (para o container; os dados ficam nos volumes).
> Se o Ambari deixar de responder (`.\lab.ps1 status` / `./lab.sh status` trava), execute `docker restart sandbox-hdp`. Em cerca de três minutos tudo volta, sem perda de dados.

7.3 Sinais de falta de memória

- Um serviço fica vermelho no Ambari logo depois de ligar: veja `lab status` (coluna de memória por processo) e o log em `/var/log/<serviço>`.
- Mensagem `Container killed on request. Exit code is 137` em um job do YARN: aumente o `swap` no `.wslconfig` ou use um perfil menor (`7a` em vez de `7`).
- O Docker Desktop trava o Windows: o valor de `memory` no `.wslconfig` está alto demais para a máquina.

8. Problemas comuns

Sintoma	O que fazer
`hdfs`, `hive`, `pig` ou `spark-submit` falham com “Connection refused”, “SafeMode” ou ficam presos	Os serviços do perfil ainda não terminaram de subir (ou o `aula` não foi executado). Espere a mensagem `PRONTO` do `aula <perfil>`; confira com `status` e no Ambari (tudo verde).
`.\lab.ps1` não roda (“execução de scripts foi desabilitada”)	Execute uma vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` no PowerShell.
`docker: error during connect` ou “Docker Desktop is not running”	Abra o Docker Desktop e espere o ícone ficar estável. No Windows, confira o WSL2 com `wsl --status`.
Ambari não abre em `localhost:8080` após alguns minutos	Veja `.\lab.ps1 logs` (ou `./lab.sh logs`). Se a porta 8080 estiver em uso por outro programa, troque-a no `docker-compose.yml` (por exemplo `"8081:8080"`).
Serviço fica vermelho no Ambari ao ligar	Ambari > Background Operations > clique na operação e leia o `stderr`. Causas comuns: memória (capítulo 7) e disco cheio do Docker (`docker system df`; limpe com `docker system prune`).
`lab aula` falha com “requisição falhou”	Repita o comando: o Ambari às vezes falha no primeiro start do NameNode após um restart. Persistindo, `lab stop-all` e tente de novo.
HDFS em “safe mode”	Aguarde um minuto. Se continuar: `sudo -u hdfs hdfs dfsadmin -safemode leave` dentro da sandbox.
`Permission denied` ao gravar no HDFS	Você está como `root`. Use `su - maria_dev` ou o terminal web com `maria_dev`.
Esqueci a senha do `admin` do Ambari	É `admin`. Se a trocou e perdeu: `ambari-admin-password-reset` dentro da sandbox, como root.
Spark reclama da versão do Python	Em shells de login a variável `SPARK_MAJOR_VERSION` escolhe o Python certo. Em um shell aberto por `docker exec` sem `-l`, execute `source /etc/profile.d/spark-python.sh`.
Mongo ou Cassandra não conectam em 127.0.0.1	Os containers precisam estar de pé: `.\lab.ps1 nosql` (ou `./lab.sh nosql`). O Cassandra leva cerca de um minuto para aceitar conexões.
Mac com chip Apple está muito lento	Confirme “Use Rosetta for x86_64/amd64 emulation” no Docker Desktop. Prefira os perfis leves (10, 11, 12, trabalho). Para a Atividade Prática o Colab continua sendo a opção mais leve.
Quero começar do zero	`.\lab.ps1 reset` (ou `./lab.sh reset`) apaga HDFS, bancos e os arquivos da `maria_dev`.

Apêndice A — Versões dos componentes

Componente	HDP 2.6.5 (vídeos)	Sandbox
Ambari	2.6.2	2.7.11 (Clemlab); sem Hive View, Tez View e Pig View; com Files View e Workflow Manager
Hadoop (HDFS, YARN, MapReduce)	2.7.3	3.3.6
Hive	1.2 / 2.1	3.1.3 (`hive` abre o beeline)
Tez	0.7	0.10.2
Pig	0.16	0.18 (terminal)
Sqoop	1.4.6	1.4.7
Oozie	4.2	5.2.1
HBase / Phoenix	1.1 / 4.7	2.5.6 / 5.1.3 (Phoenix não conecta; ver 5.2)
Kafka	1.0	3.6.2 (porta 6667 mantida; sem `--zookeeper`)
Spark	2.3 (Scala 2.11)	2.4.8 e 3.5.1 (ambos Scala 2.12; `spark.master=local[*]`)
Python	2.7	3.9 (Spark 3, mrjob) e 3.7 (Spark 2)
Zeppelin	0.7	0.11
Flume	1.5	1.11
Storm	1.1 (serviço Ambari)	2.8.9 (`lab storm start`)
Flink	1.10	1.20 (em `~/downloads`)
Drill	1.12	1.21.2 (em `~/downloads`)
Presto	0.235.1	0.235.1 (em `~/downloads`, heap 1,5 GB)
MongoDB	3.2 (serviço Ambari)	4.4 (container `mongo`)
Cassandra	3.0	3.11 (container `cassandra`)
MySQL	MySQL 5 (root sem senha)	MariaDB 10 (`root`/`hadoop`)
Sistema operacional	CentOS 7	Rocky Linux 9 (`dnf`; `yum` continua como atalho)

Apêndice B — Errata dos materiais originais

Os arquivos em `aulas/` já vêm com estas correções aplicadas. A lista serve para quem comparar com o material original dos vídeos.

Onde	Antes	Agora
Kafka	`--zookeeper host:2181`	`--bootstrap-server localhost:6667`
Kafka Connect	`kafka-standalone.sh` (erro no material)	`connect-standalone.sh`
Flink	`start-local.sh`; `jobmanager.web.port`	`start-cluster.sh`; `rest.port` em `conf/config.yaml`
Sqoop	`--driver .com.mysql.jdbc.Driver` (ponto a mais)	`--driver com.mysql.jdbc.Driver`
Spark + Mongo	`mongo-spark-connector_2.11:2.0.0`	`mongo-spark-connector_2.12:2.4.4` (Spark 2) ou `_2.12:10.4.1` (Spark 3)
Spark + Cassandra	`datastax:spark-cassandra-connector:2.0.0-M2-s_2.11`	`com.datastax.spark:spark-cassandra-connector_2.12:2.5.2` (Spark 2) ou `_2.12:3.5.1` (Spark 3)
Spark + Flume	`spark-streaming-flume_2.11:2.0.0`	`spark-streaming-flume_2.12:2.4.8` (somente Spark 2)
Pig	`ORDER ... BY numRatings AS numRatings DESC`; `videoRealese`; `numRatigs`	`ORDER ... BY numRatings DESC`; `videoRelease`; `numRatings`
Phoenix (DDL)	`PRIMARY_KEY (state,city)`; `CONTRAINT`; parêntese faltando	`PRIMARY KEY (estado, cidade)`; `CONSTRAINT`; DDL completo
Presto	`connector.name=cassadra`; `bin/laucher`; `limited 10`; `where ratings = 5`	`cassandra`; `launcher`; `limit 10`; `where rating = 5`
Cassandra (CQL)	`{'class'; 'SimpleStrategy'`	`{'class': 'SimpleStrategy'`
Aula 10, SparkSQL	script lia `customer-order.csv`, arquivo chamava `customer-orders.csv`	arquivo renomeado para `customer-order.csv` (como na aula 11)
Oozie `job.properties`	espaço sobrando no fim de `queueName=default`	removido
Hive View “Upload Table”	interface do Ambari	tabelas criadas por `lab aula 8a` (ou beeline/Zeppelin)
Storm / MongoDB no Ambari	Service Actions > Start	`lab storm start` / `.\lab.ps1 nosql` (`./lab.sh nosql`)

Apêndice C — O que é igual e o que é diferente da VM HDP

Igual

- Ambari em `localhost:8080`; usuários `admin/admin`, `raj_ops/raj_ops`, `maria_dev/maria_dev`.
- Terminal web em `localhost:4200`; SSH `maria_dev@localhost -p 2222`.
- Host `sandbox.hortonworks.com`; HDFS `hdfs://sandbox.hortonworks.com:8020`; ResourceManager na porta 8050.
- Home `/home/maria_dev`; `/user/maria_dev` no HDFS com o `ml-100k` já carregado.
- Layout `/usr/hdp/current/<componente>` (atalho para `/usr/odp/current`).
- `export SPARK_MAJOR_VERSION=2` para o Spark 2.
- Files View no Ambari; MySQL/MariaDB `root/hadoop` com o banco `movielens`.

Diferente

Assunto	VM HDP 2.6.5	Sandbox
Distribuição	Hortonworks HDP (descontinuada)	ODP 1.2.4 (Clemlab) + Ambari 2.7.11
Sistema	CentOS 7, repositórios desativados	Rocky Linux 9
Hive View, Tez View, Pig View	Existiam	Removidos ou sem suporte; use beeline, Zeppelin e `pig` no terminal
Storm, Flume, MongoDB	Serviços do Ambari	Storm e Flume à parte (`lab storm start`); Mongo e Cassandra em containers ao lado
Ligar serviços	Service Actions > Start (até 60 min para tudo)	`lab aula <perfil>` liga só os da aula (ou pelo Ambari, como no vídeo)
Python	2.7	3.9 (e 3.7 para o Spark 2)
Tempo de boot	30 a 60 min (12 h em Apple Silicon)	2 a 4 min
Onde ficam os dados	Dentro do disco da VM	Volumes do Docker (apagáveis com `.\lab.ps1 reset` ou `./lab.sh reset`)
Ambari Metrics, Ranger, Knox, Atlas, NiFi	Presentes (pesados)	Não instalados (não são usados nas aulas)